MarketBrief Architecture Proposal: Deha Tanaci

Personal Morning Market Report on Telegram using n8n

Stack: n8n (Cloud for the live bot; Docker Compose self-hosting with PostgreSQL in the repo), Telegram, Yahoo Finance, Alpha Vantage, Frankfurter, ExchangeRate-API, CNBC / BBC / Google News RSS, Supabase (managed PostgreSQL), OpenAI model via n8n AI credits · **Target:** Level 3 (project 20 of the list)

22 October 2026

1 The Need

People who follow a handful of stocks open three or four apps every morning: a broker app for their own stocks, a currency converter, and a news site, and they still have to work out what matters for *their* positions. Built-in alerts watch one stock at a time and never give the full picture in one read. **MarketBrief** is a Telegram bot where a user types `/watch tesla, airbus, turkish airlines` once, and then every weekday at 8:00 (Paris time) receives one message: the three US indices, their own stocks sorted by how much they moved, the dollar/euro rate, the four most relevant financial and political headlines, and a two-to-three sentence summary written by AI and **fact-checked by code** before it is sent.

The bot only reports and never advises ("information only, not financial advice"), so a wrong or late report costs nobody money, which keeps the project inside the course's "boring failure" rule.

> **Relation to Lab 1.** Lab 1 built a bot that *answers questions* about stocks: the user asks, the AI looks things up, the bot replies and forgets. MarketBrief is the system around that data: it **comes to the user** on a schedule, **remembers** each person's list and every report it sent, serves **many users** from one data fetch, and **keeps delivering when a source fails**. It deliberately has no `/price` or `/news` question-answering, and the AI is one small, checked step, not the engine.

2 The Triggers

The system has three entry points, and every one of them can be fired on demand:

- **Telegram Trigger** fires on every message to the bot: `/start`, `/watch`, `/unwatch`, `/list`, `/report`, `/status`, `/stop`, and the owner-only `/simulate`. For repeatable tests, n8n's test mode replays a saved Telegram message without anyone typing it.
- **Schedule Trigger** runs the morning report every weekday at 08:00, with the workflow time zone set to Europe/Paris (the course warns that time zones cause more scheduling bugs than anything else).
- **Manual Trigger** ("Run Now (Test)") is wired to the same path as the schedule, and `/report` runs the same pipeline for one person, so nobody waits for 8:00 during building or the demo.

3 Architecture & Data Flow

The project is three workflows plus two sub-workflows, sharing one Supabase PostgreSQL database with six tables. The diagram at the end shows how they connect.

Workflow A: Telegram Commands (Telegram Trigger)

1. **Telegram Trigger** → the message text, chat ID and first name.

2. **Parse And Validate Command** (Code) → recognises the command, splits company names on commas and tickers on spaces, rejects malformed input, and marks the owner. A stray "hello" or an empty `/watch` goes to a usage message and never reaches the database.
3. **Route Command** (Switch) → one branch per command, plus a fallback for anything unknown.
4. **/watch** → reads the user's list, skips what is already followed, enforces the **30-stocks-per-person cap**, calls sub-workflow **D** to turn names into tickers, saves the found ones, and replies with what each term was understood as ("tesla → TSLA (Tesla, Inc.)").
5. **/report** → checks the daily limit (5 on-demand reports per person, because each one costs an AI call and data-source quota), replies "Preparing your report…" **immediately**, and starts workflow **B** in the background without waiting for it (asynchronous).
6. **/stop** → deletes the person's stocks, subscription and report history. **/status** → shows the health of each data source and the last report. **/simulate** → owner-only switch that makes a chosen source fail on purpose, for the live demo.
7. **Send Reply** (Telegram) → one shared reply node at the end of every branch.

Workflow B: Build & Send Reports (Schedule, Manual, or called by /report)

8. **Read** source health, active subscribers, all watchlists and today's report log (PostgreSQL, retried if the database is slow).
9. **Pick Recipients** (Code) → for scheduled runs, skips anyone who already received today's scheduled report, so running the job twice **never sends twice**. Builds one combined list of symbols, so ten users following Tesla cost one price request, not ten.
10. **Get Market Quotes** (sub-workflow **C**) → every price in one call.
11. **Fetch FX Rates** (Frankfurter, with day-over-day change) and **Fetch FX Backup Rate** (ExchangeRate-API, rate only).
12. **Read CNBC, BBC Business and political headlines** (three RSS feeds), each allowed to fail on its own.
13. **Assemble Market Data** (Code) → each section (stocks, currencies, headlines) is marked ok, partial, backup or down **independently**; headlines are de-duplicated, limited to the last 36 hours and to four per source.
14. **Write Market Summary** (Basic LLM Chain + Structured Output Parser) → **one AI call per run for everyone**, never one per user. The AI receives numbers already computed by code and returns a short summary plus the IDs of the four most relevant headlines.
15. **Fact-Check AI Summary** (Code) → every number in the AI text must match a real figure from our data, advice words (buy, sell, should, recommend) are forbidden, and the length is capped. If anything fails, or the AI times out, a plain summary written by code is used instead.
16. **Compose Reports** (Code) → one personal message per recipient, with a "live prices" or "prices as of US close" line and a ⚠ note for every section that used a backup or is unavailable.
17. **Send Report** (Telegram) → **Save Delivery Log** and **Save Price History** (PostgreSQL).
18. **Check Delivery** → if reports were due but none was delivered, **Fail Loudly** (Stop And Error) so the error workflow alerts the owner, because a run that did nothing must not look green.

Sub-workflow C: Get Market Quotes

19. **Plan Quote Requests** → decides whether the primary source may be used. If Yahoo failed in **three runs in a row**, it is skipped for 30 minutes and the backup is used directly (a circuit breaker).

20. **Fetch Yahoo Quote** → Retry On Fail (2 tries), then **Fetch Alpha Vantage Quote** for anything still missing (indices through the SPY, QQQ and DIA funds). Alpha Vantage answers "200 OK" even when its daily quota is used up, so success is judged on the content, not the status code.
21. **Compute Source Health** → **Update Source Health** (PostgreSQL): counts failures in a row per source; any success resets the count.

Sub-workflow D: Resolve Stock Symbols

22. For each term: if it looks like a ticker, check it on Yahoo; otherwise use Yahoo's free company search. **Only if the search finds nothing** (e.g. "turkish airlines") is the AI asked, once for all unresolved names, and **every AI answer is re-checked on Yahoo** before it can be saved. This follows the course rule: never use AI for what a lookup can do, and never trust its output unchecked.
23. A 404 from Yahoo means the user typed something that does not exist (rejected); a timeout or 5xx means Yahoo is the problem (the user is told to retry, nothing wrong is saved).

Workflow E: Error Alerts (Error Trigger)

24. **Error Trigger** → fires when A, B or C fails after its retries (set as the error workflow of each).
25. **Notify Owner On Telegram** (workflow, failing step, error text, link to the execution) and **Write Error Log** (PostgreSQL).

4 Level Aim & Cut Line

Targeting **Level 3**, because the design is judged on what happens when a dependency times out, returns an error, or the same job fires twice. The build followed the course advice: a working Level 2 version first, then each Level 3 layer on top, so an unfinished attempt would still have demoed as a working system.

Build order

1. Level 2 core: `/start`, `/watch`, database, scheduled report to Telegram.
2. Each section fails on its own; delivery log.
3. Never-send-twice check; error workflow; fail loudly.
4. Backup sources for prices and currencies; retries.
5. AI summary with code fact-check and template fallback.
6. Circuit breaker, `/simulate`, `/status`.
7. Company names in `/watch` (sub-workflow D).

Cut line: what goes first if time runs short

1. Company-name lookup (users type tickers instead).
2. Circuit breaker (retries and backup remain).
3. `/status` and price history.
4. Shrink the AI to a fixed one-call summary with no headline picking. The AI node stays, since it is a required node type.

Never cut: input validation, database, scheduled delivery, never-send-twice, error workflow. **Status: everything above was built and tested; nothing was cut.**

5 Minimum Technical Requirements Mapping

Requirement	Where it is used
If or Set node	If: "Anyone To Send To?", "AI Available?", "Nothing Delivered?", "Under Daily Limit?", "Needs Backup?". Set: run settings, replies ("Preparing your report…").

Code node	Command parsing and validation, recipient picking, section assembly, AI fact-check, report composition, source-health computation.
Router or If/Switch	Switch “Route Command” with eight commands plus an “unknown” fallback.
API connection	HTTP Request to Yahoo Finance (prices, search), Alpha Vantage (backup, key stored as a credential), Frankfurter and ExchangeRate-API (currencies); RSS Read for three news feeds.
Message trigger	Telegram Trigger (not Chat Message).
Database for persistence	Supabase, a managed PostgreSQL database, reached with n8n’s Postgres node: six tables with keys and rules the database enforces itself, see section 6.
AI node	Basic LLM Chain with an OpenAI chat model and a Structured Output Parser (summary); a second chain in sub-workflow D for unknown company names.
Sub-workflow	“Get Market Quotes” and “Resolve Stock Symbols”, called with Execute Sub-workflow; workflow B is also started as a background sub-workflow by `/report`.
Error handling and retries	Retry On Fail on every network call; error outputs to backups and templates; circuit breaker; Stop And Error; a separate Error Workflow that alerts the owner.

6 Data Model (State, in Supabase PostgreSQL)

Table	Columns	Purpose
mr_subscribers	chat_id, first_name, active, joined_at	Who receives the morning report.
mr_watchlist	chat_id, symbol, name	Each person’s stocks (max 30). Saved name is reused when the backup source has none.
mr_report_log	chat_id, report_date, trigger, status, sections_down, summary_by, sent_at	Never-send-twice check, daily `/report` limit, `/status`, and evidence for the demo.
mr_market_history	report_date, symbol, price, change_pct, source	One row per symbol per day (upserted, so reruns do not duplicate).
mr_source_health	source, consecutive_failures, last_status, last_checked, simulate_down	Circuit breaker, `/status`, and the `/simulate` demo switch.
mr_error_log	workflow, node, message, created_at	Written by the error workflow.

Rules the database enforces. A person cannot follow the same stock twice (unique chat_id + symbol). Each person can have only one scheduled report logged per day, so even a buggy rerun cannot record a second one. Deleting a person deletes their stocks automatically (foreign key with cascade). Row-level security is switched on for every table, so the tables cannot be read through Supabase’s public web API: only n8n’s own database login reaches them.

Which copy counts. The tables are the single source of truth for who follows what; Yahoo and the other services are only read, never trusted to remember anything. **Deletion.** `/stop` removes every row about that person in the three tables that hold personal data.

7 Key Design Decisions

Decision	Why
Each report section succeeds or fails on its own	A dead price source should cost the user one section, not the whole morning report.
`/report` answers first, works after	The user should not stare at a silent chat for 15 seconds; the slow part (fetching, AI) runs in the background and delivers when ready.
One AI call per run, numbers computed by code	Cost stays flat as users grow, and the AI is used only for judgement on text (summarising, picking headlines), never for arithmetic.
Lookup first, AI last, AI checked	Yahoo search is free, instant and deterministic; the AI only handles the names search cannot, and cannot save a ticker that does not exist.
Retry temporary failures, not user mistakes	Timeouts and 5xx are retried and then routed to a backup; a 404 on a ticker is the user's typo and is rejected immediately.
A real database that enforces its own rules	The important rules (no duplicate stocks, one scheduled report per person per day, a deleted person leaves no stocks behind) live in PostgreSQL itself, so a mistake in a workflow cannot break them. The data also survives if the n8n account is lost.
Quota-aware backup	Alpha Vantage allows 25 requests a day, so it is only called for symbols the primary could not serve, one every 2 seconds, because its free plan also refuses requests sent too close together.

8 Failure Scenarios & Test Plan

Each scenario can be forced on demand (most with `/simulate`), so every one can be shown live. All were run on 21 October 2026 against the live system. After the move from n8n's built-in tables to Supabase PostgreSQL the same evening, the data was copied over and the delivery, never-send-twice, error-alert, bad-input, `/simulate`, backup-source and company-name tests were all run again on the live bot.

Scenario	How it is triggered	Expected behaviour	Result
Bad input	`hello`, `/watch` alone, `/watch qwertyzzz, TSLA`	Usage or "not understood" message; unknown name reported as not found; nothing wrong written.	**Passed**
Stranger tries the demo switch	`/simulate yahoo down` from another account	Treated as an unknown command; nothing changes.	**Passed**
Primary price source down	`/simulate yahoo down`, then `/report`	2 retries, then Alpha Vantage; report delivered with a ⚠ "prices came from the backup" note.	**Passed**
Both price sources, news and AI down	`/simulate` on alphavantage, news, ai; then `/report`	Report still delivered with what is left (currencies, template summary) and a note per missing section.	**Passed**
Currency source down	`/simulate fx down`, then `/report`	Backup rate used; note that the daily change is unavailable.	**Passed**
AI unavailable or wrong	`/simulate ai down`; AI writing a number not in the data	Summary replaced by a plain template written by code.	**Passed**

Job runs twice	“Run Now” twice in a row	First run sends; second finds today’s report already sent and sends nothing. The database also refuses a second scheduled log row for the same day.	**Passed**
Real failure in production	A live run that fails on purpose (Stop And Error)	Error workflow sends the owner a Telegram alert and logs it.	**Passed**
Name not a ticker	`/watch tesla, airbus, turkish airlines`	TSLA and AIR.PA via Yahoo search; THYAO.IS via AI, re-checked on Yahoo and flagged “matched by AI”.	**Passed**

Testing also caught and fixed five real bugs before the demo: the fact-check read the “500” in “S&P 500” as an invented number; it rejected normal rounding (1.1% for 1.05%); company names disappeared when the backup source was used; one news site crowded out the others; and the backup price source refused requests sent less than about a second apart, so its calls are now spaced two seconds apart.

9 Known Limitations

- **Yahoo Finance is unofficial.** It can change without notice; that is exactly why a backup source, a circuit breaker and per-section failure exist.
- **The fact-check verifies numbers, not direction.** If the AI wrote “fell 1.5%” for a 1.5% rise, the check would pass. A next step would compare the verb with the sign.
- **The database is one more service that can fail.** Reads are retried; if Supabase stays unreachable, a scheduled run fails loudly and the owner is alerted, because without the database the system cannot know who to send to. Price fetching keeps working without it.
- **Free database plan.** It has no automatic backups and pauses after a week without activity (the daily report prevents that). A paid plan adds daily backups.
- **One environment.** Building and the live bot share one instance; the course recommends a separate test instance before real users.

10 System Architecture & Workflow Graph

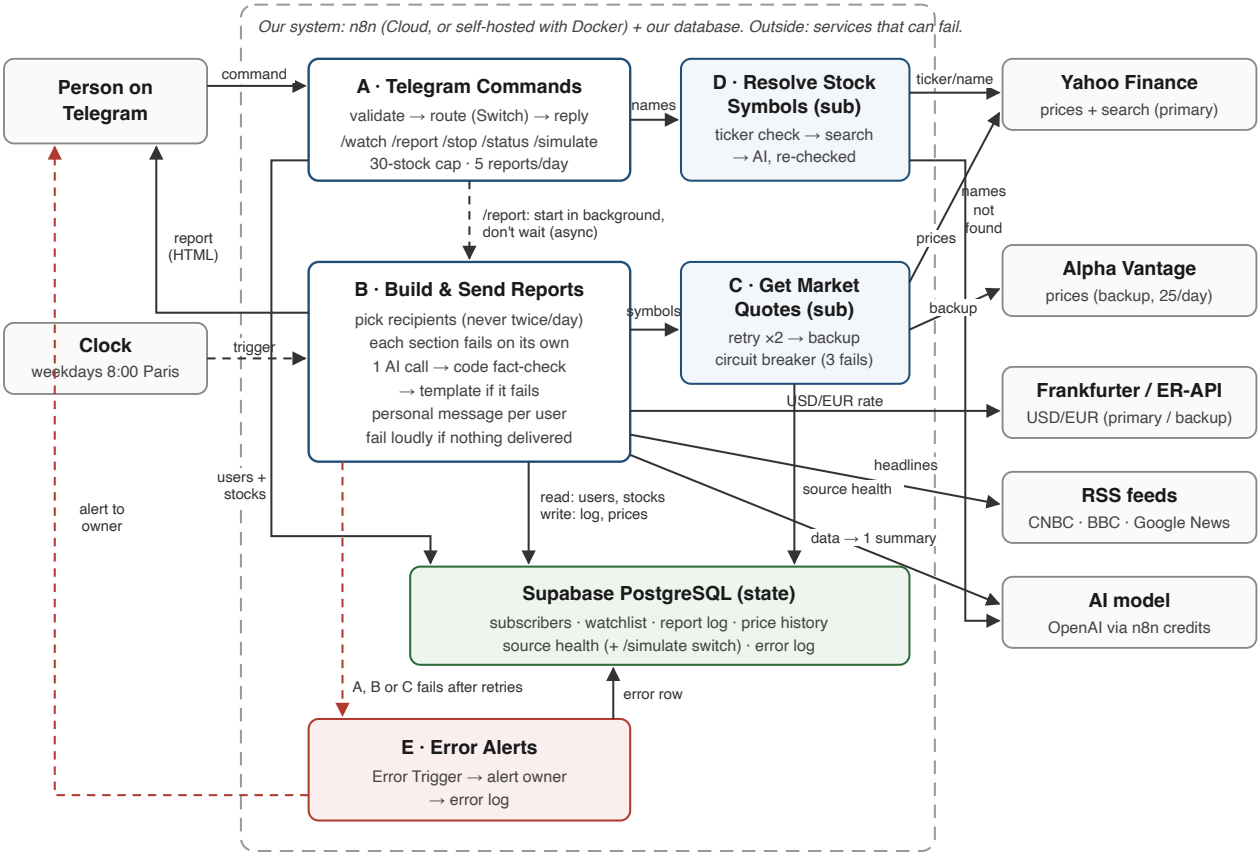


Figure 1: MarketBrief architecture: three workflows, two sub-workflows, one Supabase PostgreSQL database. **Solid arrows**: the caller waits for the answer (synchronous). **Dashed arrows**: fire-and-forget (asynchronous): the clock and `/report` start workflow B without waiting, and failures reach E automatically. Arrows point in the direction of the request; the answer returns along the same line. Every service to the right of the dashed boundary can fail; each one has a retry and either a backup or a clearly labelled “unavailable” section.